SOFTWARE DESIGN PLANE L2

From Spec-Driven Development to Intent-Driven Software Synthesis

A technical vision for architecture search, independent evaluation, shadow verification, and continuous design

Jagadesh Babu Munta

White Paper · August 2026

Humans govern intent. Machines explore designs. Independent evaluators challenge them. Evidence determines what is safe to promote.

Version 1.0

Abstract

AI-assisted software development is moving from code completion toward agentic implementation and spec-driven development. Modern systems can transform requirements into technical designs, implementation plans, code, tests, and pull requests. This paper asks a deeper question: what abstraction comes after specifications?

We propose Software Design Plane Level 2 (L2): a software-engineering control plane in which humans primarily declare capabilities, constraints, invariants, policies, and measurable outcomes, while machines synthesize and evaluate multiple candidate architectures before implementation. The key distinction is not simply AI-generated architecture; current spec-driven systems already generate technical designs from requirements. L2 instead treats architecture as a searchable intermediate representation, couples generation with independent evaluation, and connects design decisions to runtime evidence.

We introduce three conceptual abstractions: the Software Intent Graph (SIG), Design Intermediate Representation (Design IR), and Evidence Plane. Together they support a lifecycle of Intent → Synthesize → Evaluate → Verify → Operate → Observe → Re-synthesize. The paper positions L2 relative to spec-driven development, program synthesis, search-based software engineering, autonomic computing, and self-adaptive systems, and proposes an experimentally testable path toward evidence-driven architectural synthesis.

Executive Summary

- The abstraction boundary is moving upward: from code, to specifications, toward human-governed intent.
- The proposed L2 contribution is architecture-as-search-space, not merely requirements-to-design generation.
- Architecture becomes a machine-addressable Design IR that can be generated, transformed, compared, rejected, and regenerated.
- Independent evaluators assess correctness, security, reliability, performance, cost, complexity, and maintainability.
- Shadow execution produces production-like evidence before promotion.
- Runtime evidence closes the loop, enabling bounded Continuous Design rather than unrestricted autonomous self-modification.
- A minimal experiment can compare human-selected architecture, single AI-generated architecture, and multi-candidate L2 search.

The strategic layer above coding agents may become more important than any individual coding agent.

1. The Next Abstraction Boundary

Software engineering has repeatedly advanced by raising the level at which humans express intent. Machine instructions gave way to assembly, higher-level languages, frameworks, cloud abstractions,

and infrastructure-as-code. Generative AI is accelerating another shift: implementation is becoming increasingly synthesizable.

Spec-driven development (SDD) is an important current step. GitHub Spec Kit describes an intent-centered workflow whose default path is Spec → Plan → Tasks → Implement, while Kiro produces requirements, technical design, and implementation tasks from high-level feature descriptions [1][2][3].

The next question is therefore not whether AI can generate code, or even whether it can generate a technical design. The more consequential question is whether architecture can be treated as a computational search space governed by explicit intent and evidence.

2. Central Thesis: Architecture as an Output of Computation

Consider an enterprise AI inference gateway. A conventional technical specification might prescribe Go, Redis, PostgreSQL, Kubernetes, and Prometheus. An agent may generate much of the implementation, but the human has already selected most consequential architectural mechanisms.

Now express the requirement as outcomes: OpenAI-compatible inference, 10,000 sustained requests per second, less than 500 ms p99 gateway overhead, 99.99% availability, strict tenant isolation, complete auditability, zero plaintext credential storage, rolling upgrades without service interruption, and a bounded cost envelope.

The second formulation deliberately leaves the solution open. Redis, PostgreSQL, Kubernetes, Go, microservices, and serverless compute become candidate mechanisms rather than requirements.

Proposition: software architecture can increasingly become an output of computation rather than exclusively an input supplied by human engineers.

3. A Maturity Model

Level	Primary source	Human focus	Machine role	Defining shift
L0	Code	Design + implementation	Tools	Human authors implementation
L1	Prompt + code	Architecture + review	Generate code	AI accelerates implementation
L1.5	Specification	What before how	Plan + implement	Spec becomes durable source
L2	Intent + evidence	Outcomes + governance	Search + evaluate + synthesize	Architecture becomes search space
L3	Intent + runtime feedback	Policy + objectives	Re-synthesize under bounds	Continuous design

Table 1. Proposed maturity model. L1.5 reflects contemporary spec-driven workflows; L2 and L3 are proposed terms in this paper.

4. Why L2 Is Distinct from Current Spec-Driven Development

Current SDD already reaches beyond simple prompting. GitHub Spec Kit explicitly keeps intent central, supports parallel implementation exploration, and can fan workflows out and back in; Kiro's requirements-first workflow generates a technical design optimized for requirements [1][2][3]. Accordingly, L2 should not claim novelty for the generic idea of 'intent → design.'

The proposed distinction is narrower and stronger: L2 makes architectural alternatives first-class computational objects and requires evidence-driven selection across them. The core loop is multi-candidate design search → multi-dimensional evaluation → selective implementation → verification → runtime evidence.

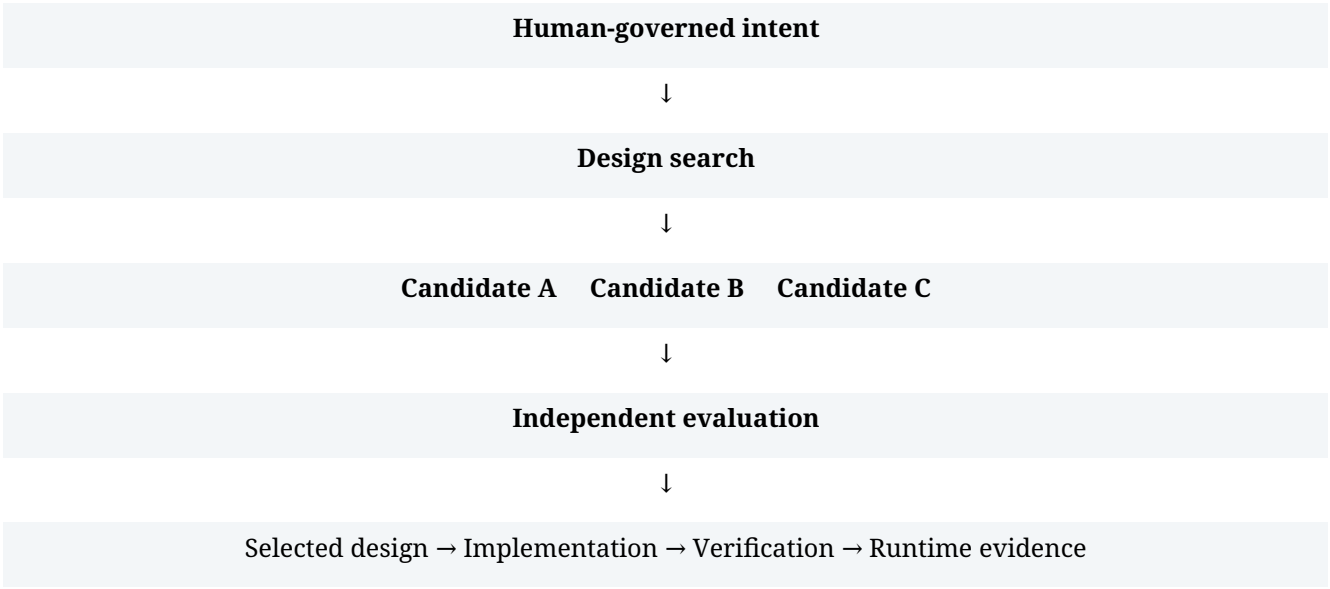


Figure 1. The L2 design-search loop.

5. Architecture as an Intermediate Representation

Compiler architecture provides a useful analogy. Compilers translate source programs through intermediate representations (IRs) that can be analyzed and optimized before target code is emitted. L2 applies the same organizing idea at system scale: Intent IR → Design IR → Implementation IR → deployable artifacts → running system.

A Design IR is not merely a diagram. It should encode components, responsibilities, interfaces, data flows, deployment topology, dependencies, policies, expected failure modes, resource assumptions, and traceability to intent. Because it is machine-addressable, it can be generated, transformed, compared, scored, rejected, and regenerated.

6. The Software Intent Graph (SIG)

The Software Intent Graph is a proposed representation connecting goals, capabilities, constraints, invariants, service-level objectives, policies, risks, design decisions, evaluators, implementation artifacts, and runtime evidence. Its purpose is bidirectional traceability: from a component back to the reason it exists, and from a requirement forward to the evidence that demonstrates whether it is satisfied.

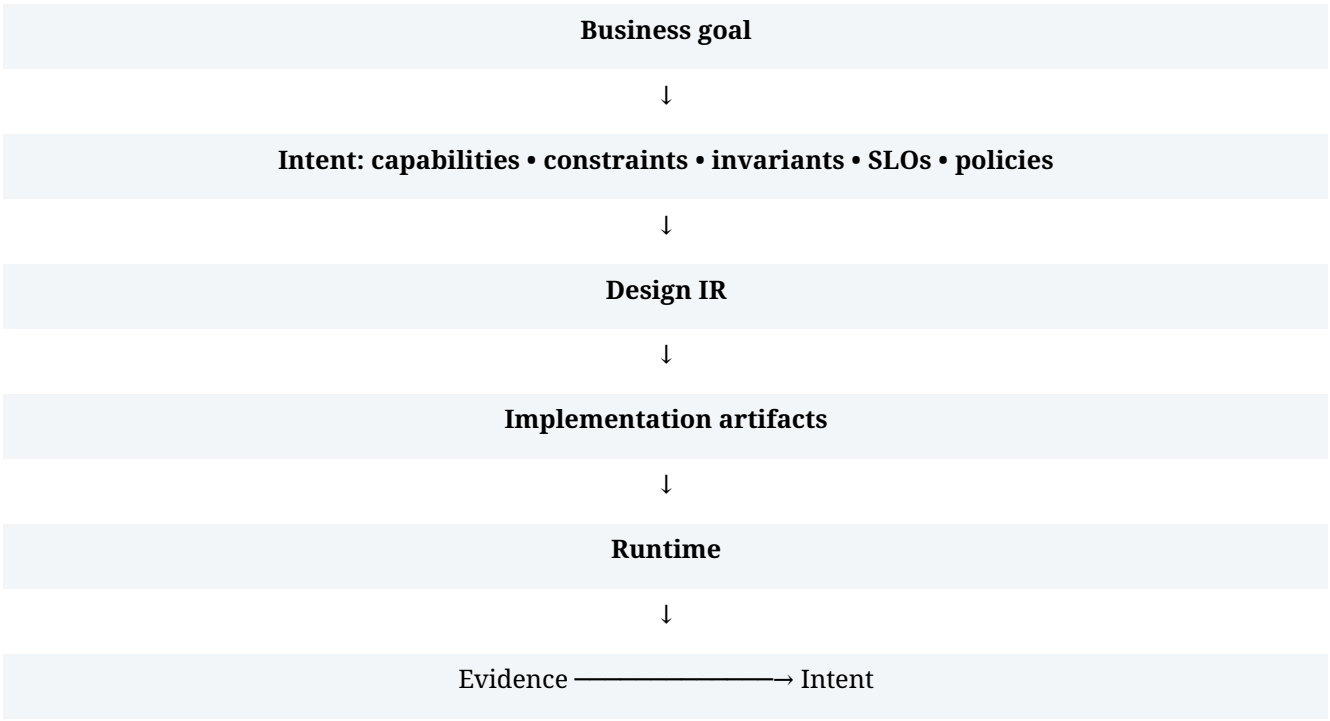


Figure 2. Conceptual Software Intent Graph.

A mature implementation should answer two questions continuously: 'Why does this component exist?' and 'What evidence demonstrates that this requirement is currently satisfied?'

7. Intent Representation

Natural language is expressive but insufficient as the sole substrate for machine-verifiable intent. A practical system would combine prose with structured declarations and executable checks. The goal is not to eliminate ambiguity entirely, but to surface ambiguity early and convert high-value requirements into measurable contracts.

```
system:
  goal: enterprise_llm_gateway
capabilities: [chat_completion, embeddings, model_routing]
scale:
  sustained_rps: 10000
slo:
  availability: ">=99.99%"
  gateway_p99: "<500ms"
security:
  tenant_isolation: strict
  plaintext_secrets: forbidden
cost:
  max_per_million_requests_usd: 20
invariants:
  - credentials_never_appear_in_logs
  - tenant_data_never_crosses_boundaries
  - every_provider_request_is_auditable
```

8. Design Search and Multi-Objective Evaluation

Search-based software engineering established that many software-engineering problems can be framed as optimization under competing constraints [6]. L2 applies this perspective to architecture-

level synthesis, where a design engine proposes a bounded set of materially different architectures rather than minor parameter variations.

Evaluation should produce an evidence vector rather than a single opaque score. A candidate may be cheaper but slower; simpler but less resilient; more secure but operationally expensive. Human policy can define hard constraints, preference orderings, Pareto frontiers, or approval thresholds.

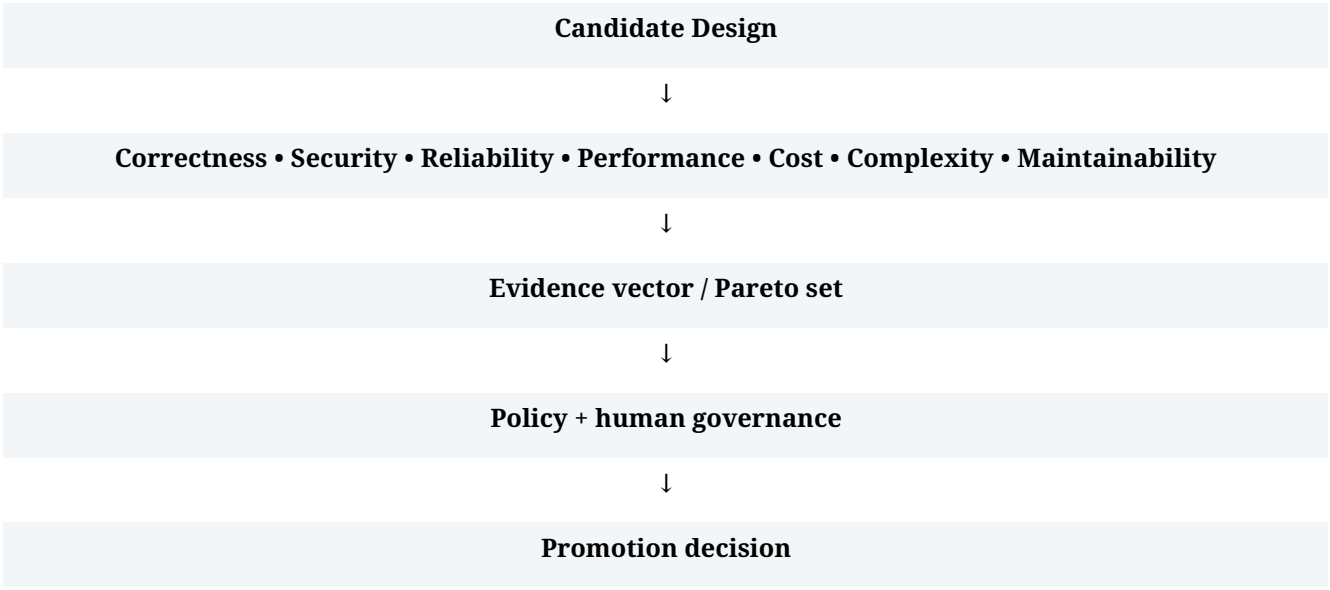


Figure 3. Evaluation should preserve trade-offs.

9. Independent Verification and the Evidence Plane

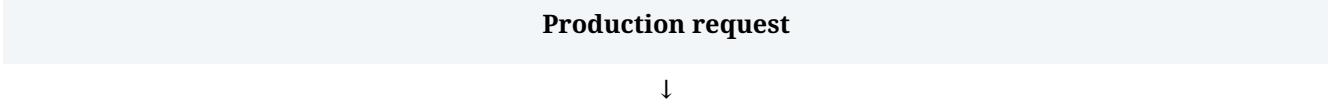
As generation becomes cheaper, evaluation becomes more valuable. The generator should not be the sole judge of its own output. Verification can combine deterministic tests, property-based tests, static analysis, policy engines, formal methods where practical, security testing, load testing, fault injection, simulations, independent model evaluators, and production evidence.

The Evidence Plane is the proposed layer that records claims and demonstrations: requirement → test result; SLO → production metric; security invariant → policy/security evidence; scalability objective → load test; reliability objective → fault-injection result; candidate design → shadow comparison.

Declared intent is not demonstrated capability. L2 continuously measures the gap between the two.

10. Shadow Execution as a Design Primitive

A candidate that passes offline tests should not automatically control user-visible behavior. Shadow execution can replay or mirror representative production workloads to a candidate while the incumbent system remains authoritative.



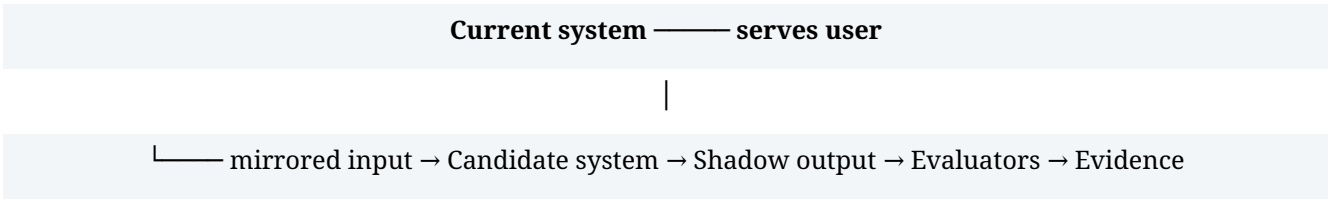


Figure 4. Shadow verification before promotion.

Promotion criteria can compare semantic correctness, latency, throughput, resource use, error behavior, policy compliance, cost, and unexpected divergence. The operating principle is: AI proposes; evaluators challenge; evidence decides; humans govern.

11. Continuous Design

Continuous integration and continuous delivery optimize the path from change to deployment. L2 adds a potential third loop: Continuous Design. When runtime evidence shows an intent violation - for example, p99 latency exceeding its declared objective - the system can initiate bounded design search, generate alternatives, verify them, shadow them, and present evidence for controlled promotion.

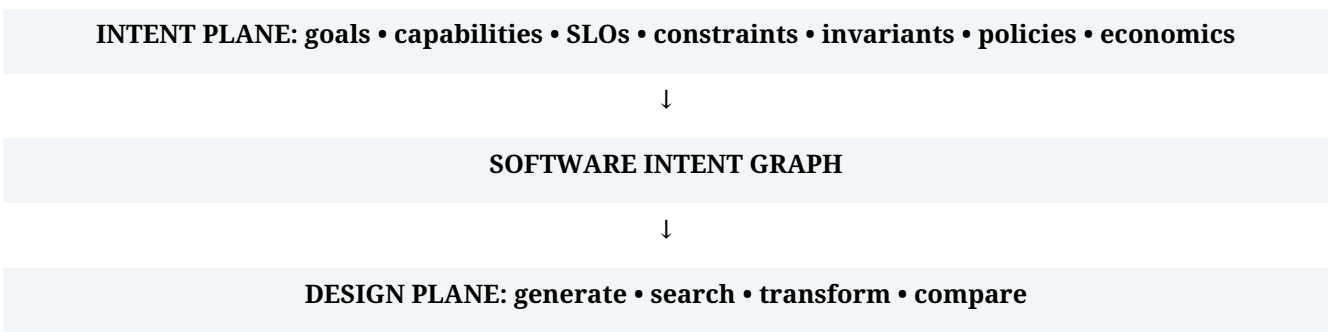
This is deliberately different from unrestricted autonomous self-modification. Intent, policy, blast-radius limits, approval requirements, and rollback mechanisms remain human-governed.

12. Relationship to Prior Work

Area	Established idea	What L2 borrows	Proposed extension
Spec-driven development	Structured intent drives design/implementation	Durable specs, guardrails, agent workflows	Architecture alternatives + evidence-driven selection
Program synthesis	Construct programs satisfying specifications	Search + verification	System/architecture-level synthesis
Search-based SE	Optimize engineering decisions under trade-offs	Multi-objective search	Architecture as a primary search object
Autonomic computing	High-level objectives guide self-management	Feedback + policy	Design-space adaptation
Self-adaptive systems	Runtime adaptation with assurances	Feedback loops, adaptation, assurance	Re-synthesis of architecture under bounded governance

Table 2. L2 is a composition and extension of established lines of work, not a claim that intent, synthesis, search, or adaptation are individually new.

13. Reference Architecture



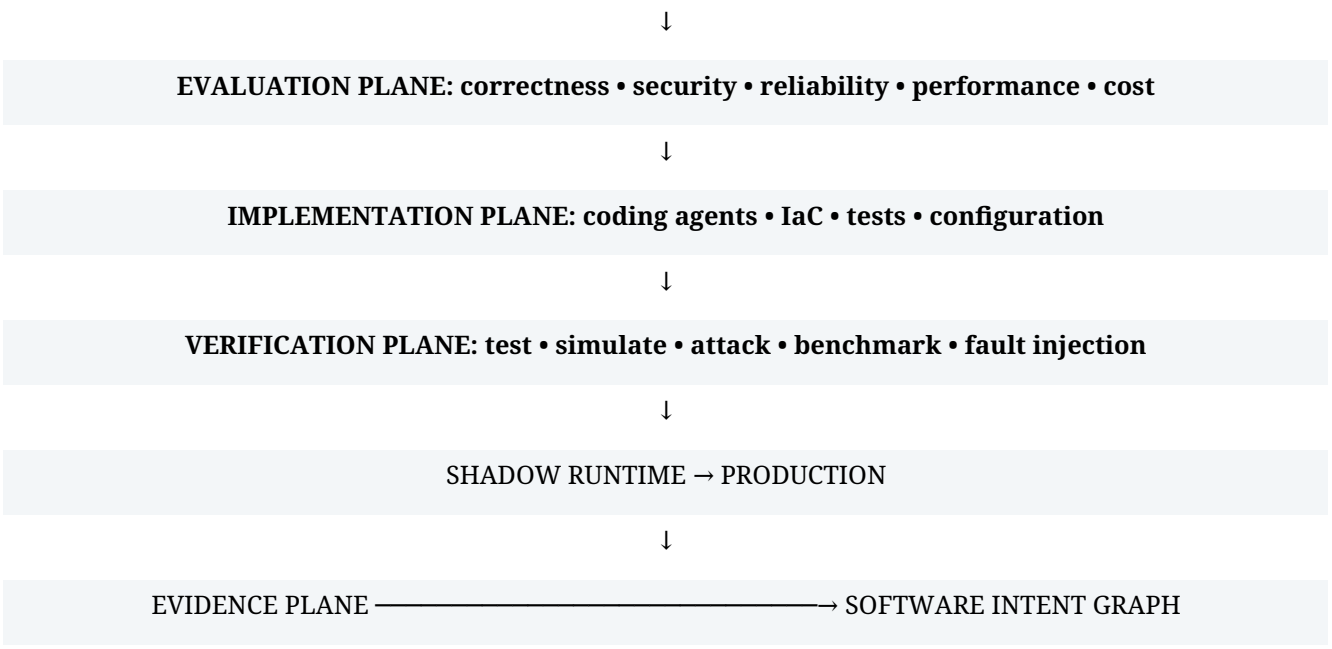


Figure 5. Conceptual L2 reference architecture.

14. A Minimal Falsifiable Experiment

The L2 hypothesis should be tested rather than treated as inevitable. A minimal experiment can define a service with explicit functional requirements, throughput, availability, latency, security, infrastructure-count, and cost constraints. Independent design agents generate three materially different architectures; each is normalized into a Design IR, selectively implemented, deployed in isolated environments, and subjected to identical tests.

- Baseline A: human-selected architecture.
- Baseline B: one AI-generated architecture.
- Treatment: L2 multi-candidate architecture search with independent evaluation.
- Measurements: percentage of requirements satisfied, p99 latency, throughput, recovery behavior, security violations, estimated cost, implementation effort, evaluator disagreement, and human review time.
- Success criterion: L2 should improve the Pareto frontier or reduce human effort without increasing unacceptable risk.

A negative result would also be informative: if architecture search produces superficial diversity, evaluators correlate too strongly, or human review cost overwhelms gains, the L2 hypothesis would require revision.

15. Research Agenda

RQ1 - Intent representation. Can human goals be made precise enough for synthesis without indirectly forcing humans to specify the solution?

RQ2 - Search efficiency. How can architecture search be bounded so that exploration is economically practical?

RQ3 - Evaluator independence. How can correlated failure modes between generators and evaluators be detected and reduced?

RQ4 - Multi-objective optimization. How should systems represent conflicts among performance, cost, security, reliability, maintainability, and complexity?

RQ5 - Evidence sufficiency. What evidence is enough to justify promotion of a generated design?

RQ6 - Safe evolution. How can architecture change while preserving state, compatibility, rollback, and operational safety?

RQ7 - Human governance. Which decisions can be delegated, which require approval, and how should accountability be recorded?

RQ8 - Design explainability. Can every consequential generated design decision be traced to intent and defended with evidence?

16. Risks, Limits, and Non-Claims

- L2 does not claim that architecture generation itself is new.
- L2 does not assume all requirements can be formalized or reduced to scalar metrics.
- L2 does not require autonomous production changes; bounded decision support is a valid and likely early form.
- LLM-based evaluators can share blind spots with generators; deterministic and heterogeneous evidence is essential.
- Architecture search can be expensive and may produce false diversity.
- Organizational, regulatory, historical, and social constraints may be only partially machine-representable.
- The terminology Software Design Plane L2, Software Intent Graph, Design IR in this specific composition, Evidence Plane, and Continuous Design as used here are proposed conceptual terms, not claims of standardized industry nomenclature.

17. Implications

For software engineers, the center of gravity moves from selecting mechanisms toward defining outcomes, invariants, risk tolerances, evaluation criteria, and approval boundaries. The architect increasingly defines the space within which acceptable architectures may exist.

For coding agents, L2 changes the strategic hierarchy. Coding agents become implementation backends beneath a design control plane that coordinates specialized generators, evaluators, runtime environments, and evidence sources.

For repositories, the long-term source of truth may expand from code to an intent repository containing goals, constraints, invariants, SLOs, Design IR, evaluation definitions, decision history, and evidence. Code remains critical, but becomes one materialization of a governed design.

18. Toward L3: Self-Evolving Software

L3 is a longer-term extension in which runtime evidence can trigger bounded re-synthesis under explicit policy. A human might tighten a latency objective while limiting cost growth; the design plane explores alternatives rather than waiting for a ticket that prescribes the mechanism. This stage requires substantially stronger assurance, governance, compatibility, and rollback mechanisms than L2 decision support.

19. Conclusion

Generative AI's ability to write code may be only the first-order effect of AI-native software engineering. Spec-driven development is already raising the abstraction boundary by making structured intent central to implementation. The next research question is whether architecture itself can become computational.

Software Design Plane L2 proposes a specific answer: represent human-governed intent, search a bounded architecture space, evaluate candidates independently, synthesize selectively, verify with multiple forms of evidence, shadow against realistic workloads, and reconcile runtime behavior against declared objectives.

The next major abstraction may not be a better programming language or even a better coding agent. It may be the design plane above them.

Working Definition

Software Design Plane L2: a software-engineering control plane that transforms human-governed intent, constraints, invariants, policies, and measurable objectives into competing architectural designs; evaluates those designs through independent evidence; synthesizes implementations; and continuously reconciles running systems against declared intent.

References

[1] GitHub. Spec Kit Documentation: Spec-Driven Development. 2026. https://github.github.com/spec-kit/

[2] GitHub. Spec Kit, 'What is Spec-Driven Development?' 2026. https://github.com/github/spec-kit/blob/main/docs/concepts/sdd.md

[3] Kiro. Feature Specs: Requirements-First and Specs documentation. 2026. https://kiro.dev/docs/specs/feature-specs/requirements-first/

[4] Kephart, J. O., and Chess, D. M. 'The Vision of Autonomic Computing.' Computer 36(1), 41-50, 2003. doi:10.1109/MC.2003.1160055.

[5] Cheng, B. H. C., et al. 'Software Engineering for Self-Adaptive Systems: A Research Roadmap.' LNCS 5525, 1-26, 2009. doi:10.1007/978-3-642-02161-9_1.

[6] Harman, M., and Jones, B. F. 'Search-Based Software Engineering.' Information and Software Technology 43(14), 833-839, 2001. doi:10.1016/S0950-5849(01)00189-6.

[7] Gulwani, S., Polozov, O., and Singh, R. 'Program Synthesis.' Foundations and Trends in Programming Languages, 2017.

[8] Alur, R., et al. 'Syntax-Guided Synthesis.' Formal Methods in Computer-Aided Design (FMCAD), 2013.

[9] Solar-Lezama, A. Program Synthesis by Sketching. PhD dissertation, University of California, Berkeley, 2008.

[10] Brun, Y., et al. 'Engineering Self-Adaptive Systems through Feedback Loops.' In Software Engineering for Self-Adaptive Systems, LNCS 5525, 48-70, 2009.

Publication Note

This paper is a technical vision and research agenda. The L2 maturity level and the specific composition of SIG, Design IR, Evidence Plane, shadow verification, and Continuous Design are proposed here as a framework for discussion and experimentation. Established concepts and systems are cited as prior work. Claims about future capability are hypotheses, not assertions of current production readiness.

Suggested Citation

Munta, J. B. (2026). Software Design Plane L2: From Spec-Driven Development to Intent-Driven Software Synthesis. White Paper, Version 1.0.

Keywords

AI-native software engineering; intent-driven development; spec-driven development; architecture search; software synthesis; Software Intent Graph; Design IR; Evidence Plane; AI coding agents; shadow execution; continuous design; self-adaptive systems; search-based software engineering.